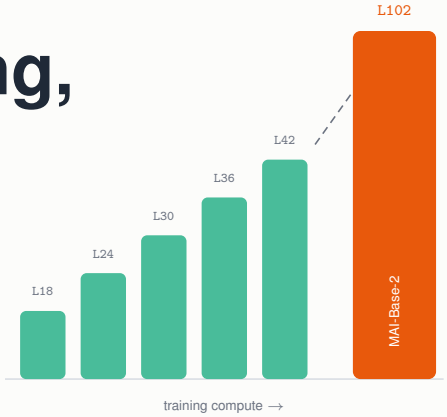
THE TRANSFORMER, VISUALLY · PART TWO

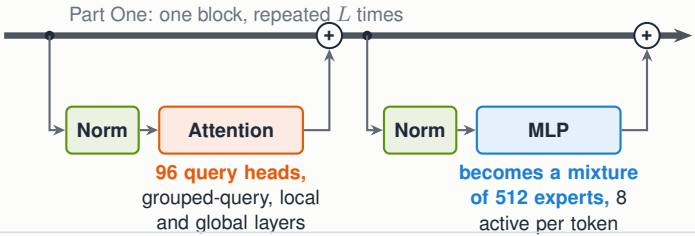
Frontier Pre-training, Visually

How MAI-Base-2 was built: scaling ladders, hyperparameter laws, mixture-of-experts systems, Muon, stability and data. A visual companion to Microsoft AI's report *Frontier Pre-training at Microsoft AI* (2026).



Where Part One ended, and what this deck adds

Part One explained the block. Part Two is about the recipe: how a frontier lab decides what to build and how to train it.



Stacked 102 times. 2.8T parameters in total, 80B active per token.

How do you choose? Scaling ladders and hyperparameter laws that predict the big run from small ones.

How do you make it fast? MoE kernels, the Muon optimizer and clipping, co-designed with the hardware.

What do you feed it? 50 trillion tokens of text and images, processed in-house.

MAI-Base-2 in numbers

A mixture-of-experts model trained on 16k GB200 GPUs over 38 days in August and September 2026.

80B / 2.8T
active / total parameters

102
layers, hidden width 8,704

50T
training tokens, text and vision

16k
GB200 GPUs, 38 days

21%
MFU, against 2.5 PFLOPS BF16 peak

89%
goodput: share of time making progress

$\approx 2.4 \times 10^{25}$
training FLOPs (derived)

4–12×
FLOPs-efficiency over the best open base

Background: do the numbers agree?

Model side. Training costs about $6\,N_{\text{active}}D$ FLOPs (Part One): $6 \times 80\text{B} \times 50\text{T} \approx 2.4 \times 10^{25}$. **Hardware side.** $16{,}000 \times 2.5\,\text{PFLOPS} \times 38\,\text{d} \times 21\% \times 89\% \approx 2.5 \times 10^{25}$.

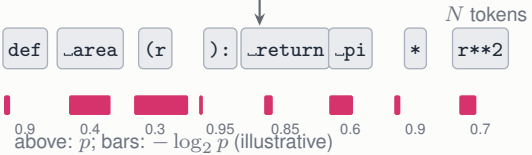
Both agree with the last point of the report's scaling ladder (Figure 2, about $10^{25.4}$). MFU excludes rematerialization and optimizer FLOPs, which is why the $6ND$ count is the right comparison.

Reading the metric: bits per byte

Every comparison in the report is held-out bits per byte: a compression score that does not depend on the tokenizer.

held-out text, B bytes of UTF-8

```
def area(r):  return pi*r**2
```



$$\text{BPB} = \frac{1}{B}\sum_{i=1}^{N} -\log_2 p(x_i \mid x_{<i}) \;=\; \frac{N}{B}\cdot\frac{\overline{\text{NLL}}}{\ln 2}$$

Tokens cancel: a tokenizer with bigger pieces has fewer tokens, each costing more bits. The total bits per byte of text is what stays comparable across models.

Background: why this metric

A compressor's score. An arithmetic coder driven by the model would compress the text to BPB bits per byte; 0.22 on code means $8/0.22 \approx 36\times$.

Held out on purpose. Code is MAI's own codebase; the STEM, Math and QA sets were commissioned from vendors, and all were decontaminated against the training data.

Caveat kept by the authors. Three comparators are post-trained checkpoints, and post-training can move these numbers.

Figure 1: held-out bits per byte, open models above 1T

Lower is better. Green is MAI-Base-1, orange is MAI-Base-2; starred comparators are post-trained checkpoints.

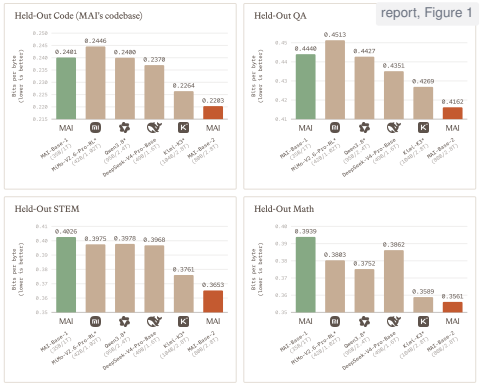


table values read from Figure 1; margins derived

Model (active / total)	Code	QA	STEM	Math
MAI-Base-1 (35B / 1T)	.2401	.4440	.4026	.3939
MiMo-V2.6-Pro-RL* (42B / 1.02T)	.2446	.4513	.3975	.3803
Qwen3.8* (95B / 2.4T)	.2400	.4427	.3978	.3752
DeepSeek-V4-Pro-Base (49B / 1.6T)	.2370	.4351	.3968	.3862
Kimi-K3* (104B / 2.8T)	.2264	.4269	.3761	.3589
MAI-Base-2 (80B / 2.8T)	**.2203**	**.4162**	**.3653**	**.3561**

Lowest on all four. Margin over the runner-up, Kimi-K3: 2.7% on code, 2.5% QA, 2.9% STEM, 0.8% math.

Against its predecessor. 8% lower on code, 6% QA, 9% STEM, 10% math, with $2.3\times$ the active parameters and $1.7\times$ the tokens.

What the gap is worth. The report's yardstick is compute: how many FLOPs an improvement saves. That is where the deck goes next.

Two models side by side

MAI-Base-2 is larger, sparser and trained longer than MAI-Base-1, and almost every line of the recipe changed.

	MAI-Base-1	MAI-Base-2
Active / total parameters	35B / 1T	80B / 2.8T
Layers, hidden width	78, 6,656	102, 8,704
MoE layout	alternating dense FFN / MoE	MoE in every layer
Dense FFN width	13,312	none
MoE down projection, expert width	3,072, 10,240	4,096, 8,704
Experts per token	0 shared + 8 of 512 routed	1 shared + 7 of 512 routed
Attention heads, KV / Q	8/80 global (1 in 6), 8/80 local	2/96 global (1 in 6), 16/96 local
Embeddings	tied	untied
Training horizon	30T tokens	50T tokens
Optimizer	AdamW	Muon (2-D weights) + AdamW
FP8 scheme	tensor-wise delayed scaling, E4M3 / E5M2	MXFP8, E4M3
Gradient clipping	global weight-gradient norm	+ activation gradients
Vision	text only	native VLM

table transcribed from the report; the shared expert is not down/up projected

Layer layout

Base-1: dense / MoE alternating



Base-2: MoE in every layer



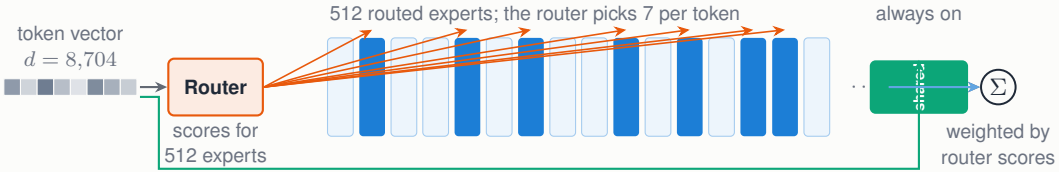
dense FFN MoE

Sparser. $80/2800 = 2.9\%$ of the weights act on each token; 3.5% for Base-1.

Why every layer. It always won on FLOPs; faster kernels made it win on wall-clock time too (MoE systems section).

Background: the MLP becomes a mixture of experts

Part One's MLP, replicated 512 times. A router sends each token to 7 of them plus one expert that everybody uses.



Active vs total. Each token runs 8 expert MLPs of width 8,704; all 512 are stored. That is how 2.8T parameters cost 80B per token.

A bottleneck in front. The 4,096-wide down projection (none for the shared expert) reads as: routed experts see a compressed copy of the stream.

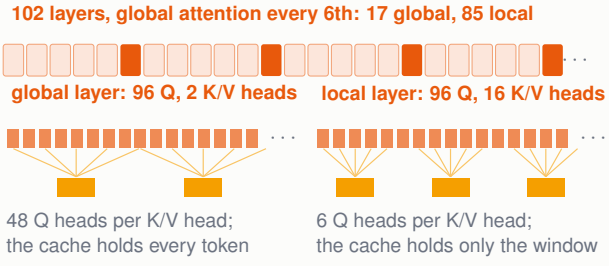
Background: where the design comes from

Sparsely gated MoE (Shazeer et al. 2017): a learned router makes a layer's cost independent of its parameter count. GShard and Switch Transformer (2020–21) scaled it, with a load-balancing loss that keeps experts evenly used.

Fine-grained and shared experts (DeepSeekMoE 2024): many small experts give the router more combinations; one shared expert carries what every token needs, so routed experts can specialize. DeepSeek-V3 uses 1 shared + 8 of 256; MAI-Base-2 uses 1 shared + 7 of 512.

Background: attention and embedding choices

Five local layers for every global one, few key/value heads where the cache is unbounded, and untied embeddings.



Reading the trade. Base-1 used 8 K/V heads in both kinds of layer. Base-2 spends K/V capacity where caching is cheap (local, 16) and saves it where every token is cached (global, 2).

Background

Grouped-query attention (Ainslie et al. 2023): query heads share K/V heads, shrinking the KV cache by the group size.

Local + global layers. Local layers keep cost linear in context length; global ones carry long-range information. Gemma 2 alternates them one to one; MAI-Base-2 uses five to one.

Untied embeddings. Tying input and output tables (Press and Wolf 2017) saves $V \times d$ parameters and acts as a regularizer. At 2.8T parameters the saving is negligible, so the two tables are free to differ.

Three initiatives, three sections of this deck

The report credits MAI-Base-2 to scaling science, model–infrastructure co-design and data. The deck follows that order.

Scaling science

Scaling ladders that account for repeated training data (*source scaling*), and empirical scaling laws for learning rate, weight decay and dropout. Figures 2, 3, 4, 5.

footer: Ladders · Hyperparameters

Model–infrastructure co-design

An efficiency-gain metric in FLOPs and in wall-clock time; multi-round dropless MoE; Muon resharded for 16k GPUs; activation gradient clipping. Figures 6, 7, 8, 9.

footer: MoE systems · Muon · Stability

Scaling data

Expanded in-house pipelines, embedding-based diversity sampling, a 50T-token mixture, and native vision data. Figure 10.

footer: Data

Scaling ladders: one number sets the whole model

A ladder is a family of models with one fixed recipe. Each rung is named by its layer count L, and L fixes the rest.

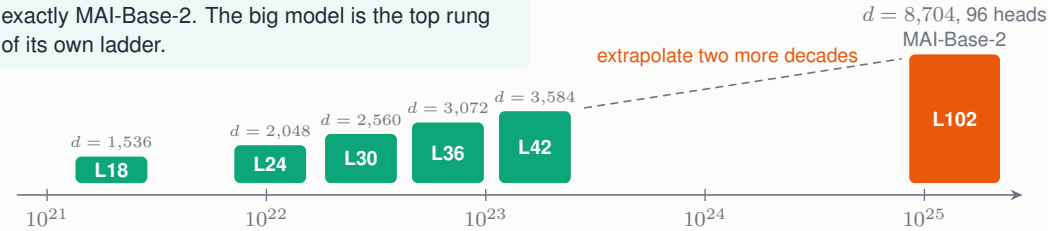
The rules of this ladder (from the report)

hidden width $d = \frac{256}{3}L$; query heads $= L$ rounded to a multiple of 16; tokens per active parameter (TPP) held fixed.

Check: $L = 102$ gives $d = 8{,}704$ and 96 heads, exactly MAI-Base-2. The big model is the top rung of its own ladder.

Why a ladder. Every design question is answered by re-running the rungs and checking that the gain holds as L grows.

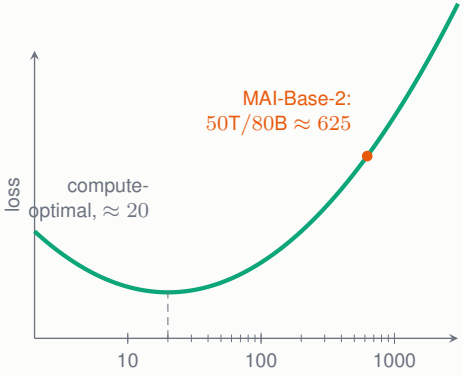
The hard part is making small rungs behave like the big model: source scaling, two slides on.



training compute (FLOPs); rung positions read off Figure 2, approximate

Background: scaling laws and tokens per parameter

Loss falls as a power law in compute. How to split compute between parameters and tokens is a separate choice.



Curve illustrative: at fixed compute, loss is lowest near 20 tokens per parameter and rises slowly to the right.

Background

Power laws (Kaplan et al. 2020): loss falls smoothly with parameters, data and compute, which is what makes a ladder extrapolable.

Chinchilla (Hoffmann et al. 2022): at a fixed training budget, grow parameters and tokens together; their fits imply about 20 tokens per parameter.

Why train far past it. Serving cost scales with active parameters, not with training tokens, so frontier models sit far to the right of the training-optimal point.

Source scaling: repeat the data alike on every rung

Small runs usually see each example once, the big run many times. This ladder repeats data alike on every rung.

full run: 50T tokens = 8 passes over 6.25T unique tokens



a 5T rung: 8 passes over 1/10 of the data

← one tenth the tokens, the same 8-fold repetition

without source scaling: 5T unique tokens, one pass



← no repetition: a different, easier problem

Fixed TPP and fixed repetition fix the unique data per parameter on every rung. Without that, the report says, small experiments favor under-regularized choices that overfit at scale.

Background: data-constrained scaling

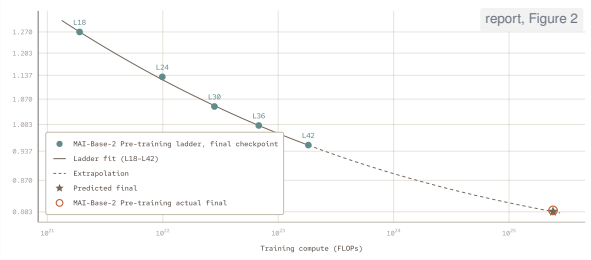
Muennighoff et al. 2023: up to about 4 passes, repeated tokens are nearly as good as new ones; beyond that their value decays toward zero. Repetition is a regime with its own scaling law.

Why small runs mislead. A single-epoch run never shows the overfitting that eight passes cause, so dropout, weight decay and width get tuned for the wrong regime.

See Figure 10: unique and repeated tokens are shown separately; repetition is part of the mixture design.

Figure 2: the ladder forecast the final model

Five rungs across two decades of compute, fit once, extrapolated two more. The prediction landed on the actual value.



What is plotted. Each rung's final-checkpoint evaluation against its training FLOPs, a curve fit through L18–L42, and its extrapolation to $10^{25.4}$ FLOPs.

The test. The star is the forecast for L102; the open circle is MAI-Base-2's actual final value. They coincide near 0.80 on the report's axis.

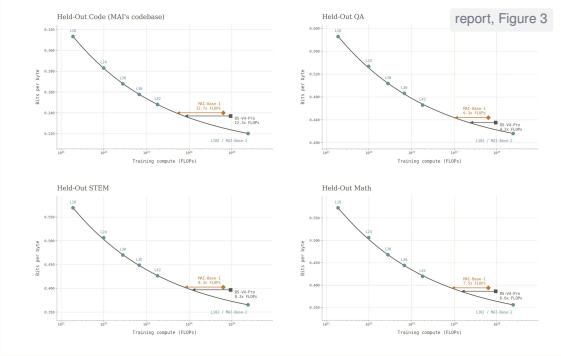
Why it matters. The decision to spend 16k GPUs for 38 days rested on a curve from runs over $100\times$ cheaper, and the curve held.

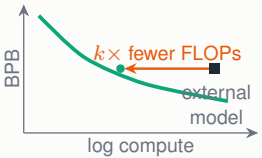
Background: what a ladder fit assumes

The usual form is a power law with a floor, $\mathcal{L}(C) = E + A\,C^{-\alpha}$ (Kaplan 2020; Hoffmann 2022); the report does not state its fit. It predicts the planned model under the same recipe and data; source scaling keeps the rungs on one curve.

Figure 3: how much compute is the recipe worth?

Project an external model onto MAI's curve. The horizontal gap is the compute the recipe saves at equal quality.





	vs DS-V4	vs Base-1
Code	12.3×	12.7×
QA	4.2×	6.3×
STEM	8.3×	8.3×
Math	6.6×	7.5×

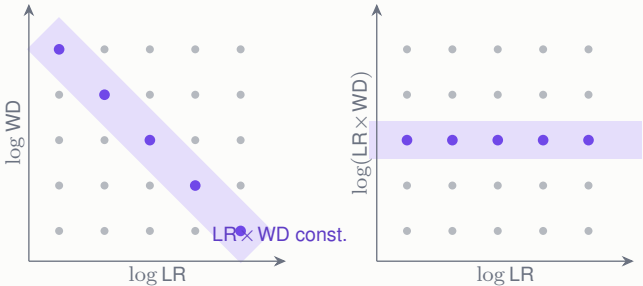
Method. Per evaluation, fit BPB against FLOPs on the ladder (Base-2's own point included), find where the curve reaches the external model's BPB, and divide. It needs the comparator's training FLOPs.

Result. 4.2–$12.3\times$ against DeepSeek-V4-Pro-Base, the strongest open base model; 6.3–$12.7\times$ against MAI-Base-1 from June, four months earlier.

LR and weight decay: one knob and a half

Good configurations lie on a diagonal of the (LR, WD) plane; in rotated coordinates the diagonal becomes an axis.

sketch of the report's argument; band position illustrative



rectangular grid: 25 runs, 7 land in the good band

rotated grid: the band is a row and every LR is probed

Background: why the product matters

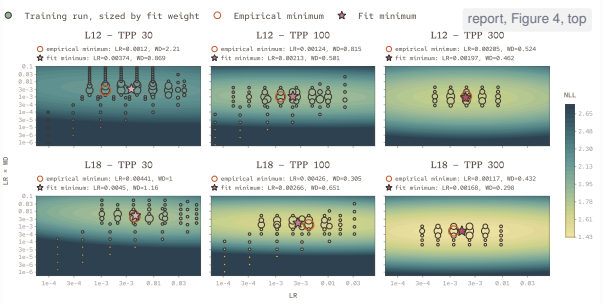
AdamW's decoupled decay (Loshchilov and Hutter 2019) multiplies the weights by $(1 - \eta\lambda)$ each step, so they act as a moving average of recent updates with timescale $\tau = 1/(\eta\lambda)$ steps (Wang and Aitchison 2024).

Two separate roles. LR sets the step size; LR×WD sets how far back the weights remember.

Scope. Base-2 trains most matrices with Muon, but the report states the sweep in LR and WD; so does the deck.

Figure 4, top: fit the loss surface, do not pick the best run

Each panel is one model size and token budget. A 2-D quadratic is fit to the runs; its analytic minimum is the answer.



fit minimum	L12, TPP 30	L12, 100	L12, 300	L18, 30	L18, 100	L18, 300
LR*	0.0037	0.0021	0.0020	0.0045	0.0027	0.0017
WD*	0.87	0.50	0.46	1.16	0.65	0.30

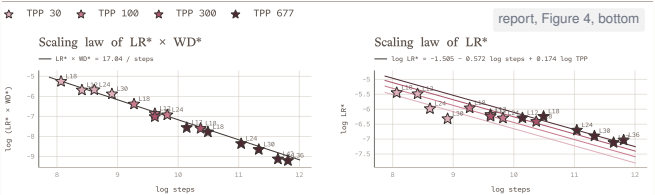
How to read a panel. Circles: training runs on the rotated grid, sized by their weight in the fit. Background: the fitted quadratic. Star: its minimum. Open circle: the best single run.

Why fit. The best run is noisy and sits on a grid point; the surface minimum interpolates between runs and describes the whole valley.

The trend. More steps (higher TPP) lower both LR* and WD*. Table: fit minima from the panel captions.

Figure 4, bottom: the laws behind the hyperparameters

Fitted optima from many (size, TPP) slices, modeled as functions of training steps and tokens per parameter.



$$\mathrm{LR}^* \times \mathrm{WD}^* = 17.04/\mathrm{steps}$$

$$\log \mathrm{LR}^* = -1.51 + 0.175 \log \mathrm{TPP} - 0.573 \log \mathrm{steps}$$

$$\mathrm{Dropout} = \mathrm{sigmoid}\big(0.526\,(\log \mathrm{ActiveParams} - 27.2)\big)$$

Model selection found $\mathrm{LR}^* \times \mathrm{WD}^*$ depends on steps alone and LR^* also on TPP. The laws were fit at small scale, validated at an intermediate scale against the old heuristics, then applied to MAI-Base-2. The unit of ActiveParams and the log base are not stated, so no dropout value is computed here.

Background: both exponents have a story

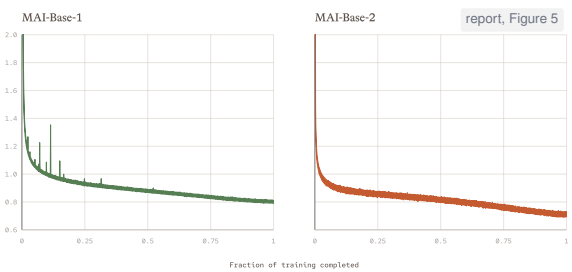
Exponent -1. $1/(\mathrm{LR} \cdot \mathrm{WD}) = \mathrm{steps}/17.04$: the weights average over the last $\approx 6\%$ of the run, however long it is. Wang and Aitchison 2024 argue for a timescale fixed in epochs, which for one pass is the same thing.

Exponent ≈ -0.5. At fixed tokens, halving the steps doubles the batch; $\mathrm{LR} \propto \mathrm{steps}^{-1/2}$ is the $\sqrt{2}$ rule, derived for Adam-type optimizers by Malladi et al. 2022.

Dropout grows with size, as expected under source scaling.

Figure 5: a run with no spikes

Training loss over the full horizon. MAI-Base-1 spiked early; MAI-Base-2 did not spike at all.



Background: where loss spikes come from

Attention or output logits blow up, or an unusual batch arrives. Wortsman et al. 2023 reproduce this at small scale (fixes: QK-norm, z-loss, lower LR at size); PaLM restarted and skipped batches. Base-2 clips the offenders at the source instead.

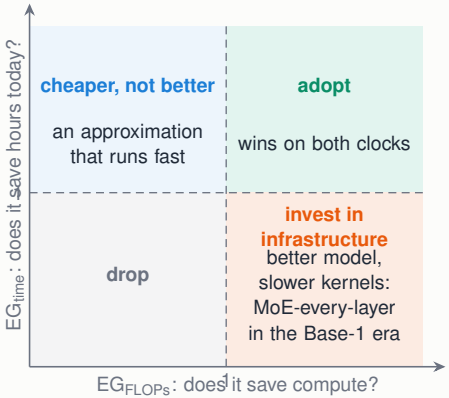
What changed. The report credits the smooth curve to the ladders and to incremental refinement; the Stability section adds two concrete pieces, Muon and activation gradient clipping.

Not comparable in level. The runs use different data mixtures (Figure 10), so Base-2's lower end point is not by itself evidence of a better model; Figure 1 is.

Why spikes matter. A spike wastes the tokens around it and often forces a rollback to an earlier checkpoint, which on 16k GPUs is costly.

Co-design: score every idea in FLOPs and in wall-clock time

Efficiency gain (EG): the compute a candidate needs to reach the baseline's loss, relative to the baseline. Two clocks.



Decisions follow EG_{time}, which reflects the infrastructure as it is. **EG_{FLOPs} is the map:** a change that wins on FLOPs but loses on time shows where to invest, and once the kernels catch up it moves to the top right.

Three results of this loop: MoE in every layer, Muon at 16k GPUs, activation gradient clipping.

Background: the two utilization numbers

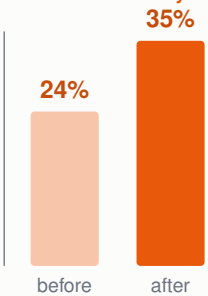
MFU (PaLM 2022): achieved tokens per second divided by the peak the hardware could do, counting only forward and backward model FLOPs. Base-2: 21% against GB200's 2.5 PFLOPS.

Goodput: the share of wall-clock time in which training actually advanced; 89% means 11% went to restarts, stragglers and checkpoints.

Scaling MoE: two inefficiencies, then MoE in every layer

MoE-every-layer always won on FLOPs. Fixing the MoE implementation made it win on time as well.

MFU of the MoE layers



$+46\%$ relative; with the same model FLOPs that is $1.46\times$ the throughput in those layers

1 Dynamic routing fragmented memory

Experts receive variable token counts; buffers sized for rare extreme imbalances tied up memory that could have cut activation recomputation.

2 Finer experts, more overhead

512 small experts mean many small GEMMs, and more launches, synchronizations and collectives per layer.

Consequence

MoE-layer MFU rose from 24% to 35%, which moved MoE-every-layer from “invest” to “adopt”. Base-1 had alternated dense FFN and MoE layers because of $\mathrm{EG}_{\mathrm{time}}$; Base-2 uses MoE in all 102.

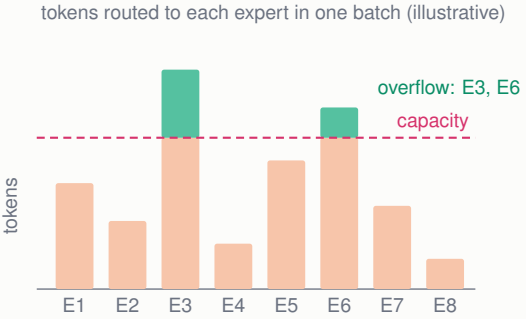
Background

Activation recomputation (rematerialization) trades memory for compute by recomputing activations in the backward pass. Memory freed from MoE buffers buys back that compute.

Fixes for (1) and (2) are the next three slides: a multi-round dropless MoE with fixed memory, and four cuts to per-expert overhead.

Background: the capacity problem in expert parallelism

Routing is data-dependent, so some experts get far more tokens than others. Every MoE system must decide what then.



Why it is hard on GPUs: memory is allocated ahead of time, but the token counts are only known after the router runs, every step, on every layer.

Drop the overflow
Capacity factor with token dropping (GShard, Switch): fixed memory, but dropped tokens skip the layer. MAI measured worse models.

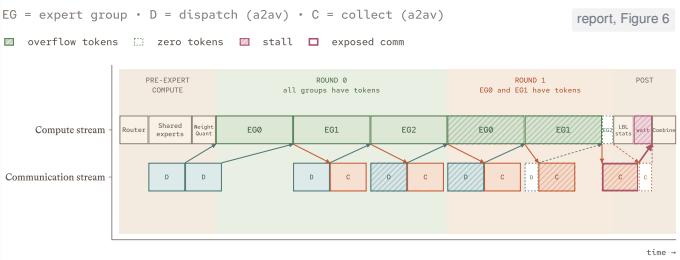
Allocate for the worst case
Buffers sized for any imbalance: never drops, but most of that memory idles under normal loads (inefficiency 1).

Dropless, variable memory
MegaBlocks (2022): block-sparse kernels process whatever arrives; no drops, but memory moves with the routing.

MAI: dropless, fixed memory
Fixed capacity per expert, overflow handled in extra rounds: nothing dropped, memory static, one round in steady state.

Figure 6: multi-round dropless MoE, one forward pass

Three expert groups; groups 0 and 1 overflow. Compute and communication streams run side by side, time left to right.



Vocabulary. EG = expert group (the experts on one rank); D = dispatch, C = collect, both all-to-all exchanges.

Pre-expert. The router runs; shared experts and weight quantization overlap the first, exposed dispatch.

Round 0. All groups have tokens. While EG0 computes, EG1's dispatch and EG0's collect run on the communication stream.

Round 1. Only EG0 and EG1 overflowed. The round count is global, so EG2 takes its turn with zero tokens and skips compute.

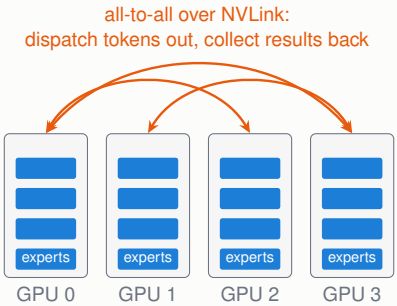
Post. Load-balance statistics, a wait, the final combine. The last collect is the only other exposed communication.

Fixed memory, and the backward pass

Each expert has a fixed token capacity; overloaded experts borrow unused capacity within their group, then take extra rounds. Capacity is set so that steady state needs one round. The backward pass recomputes per group on the same schedule, with all expert communication overlapped.

Cutting per-expert overhead: four changes

With 512 experts the work per expert is small, so launches, host round-trips and copies dominate unless removed.



Expert parallelism (GShard 2020): experts live on different GPUs; each token's hidden state travels to its experts and back. Per expert, that is a small GEMM wrapped in communication.

1 Group work per expert group

One batched GEMM and one collective per group, not per expert: fewer launches, fewer tail effects.

2 Metadata stays on the GPU

Token counts and offsets live on the device and the all-to-all is issued from the GPU: no host round-trip.

3 Symmetric memory over NVLink

Tokens move directly between GPU memories without staging copies, which also eases HBM contention.

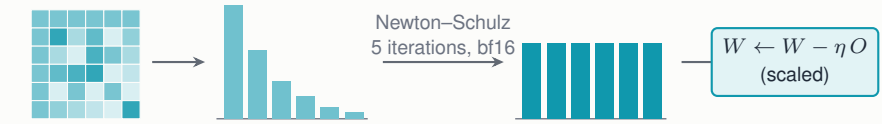
4 One load-balancing reduction

The auxiliary loss is batched across layers: one reduction per step instead of one all-reduce per MoE layer.

Report: sparser models "without significantly degrading MFU".

Background: Muon orthogonalizes the update

AdamW scales each weight separately. Muon treats the update as a matrix and equalizes its singular values.



$$X_0 = M/\|M\|_F, \qquad X_{k+1} = aX_k + b\,(X_k X_k^\top)X_k + c\,(X_k X_k^\top)^2 X_k$$

Effect. Rare gradient directions move as much as dominant ones (steepest descent under a spectral-norm bound; Bernstein and Newhouse 2024).

In MAI-Base-2: Muon for non-embedding 2-D weights, AdamW for the rest; more stable, large gains.

Background

Origin. Keller Jordan 2024; five bfloat16 Newton–Schulz steps suffice.

Scale. Kimi K2 (2025) trained a 1T-parameter MoE with MuonClip; Liu et al. 2025 show the gain holds up at scale.

Muon on 16k GPUs: reshard, orthogonalize, reshard back

ZeRO shards momentum across data-parallel GPUs; Newton–Schulz needs whole matrices. Two all-to-alls bridge them.

1 Local momentum update
each GPU updates its FP32 shard of every momentum matrix and casts it to BF16

→

2 All-to-all, batched
shards of same-shaped matrices are exchanged so each GPU now owns a subset of whole matrices

→

3 Newton–Schulz locally
a symmetry-aware kernel orthogonalizes each whole matrix on the GPU that holds it

→

4 All-to-all back
updates return to their original shards, are cast to FP32 and applied to the weights


one matrix, four shards


whole matrix on one GPU

Three more cuts. Prefetch several batches so communication overlaps compute; cache the resharding metadata (an MoE has thousands of identically shaped matrices); replicate small weights such as the routers, so step 2 is skipped and one all-gather after Newton–Schulz distributes their updates.

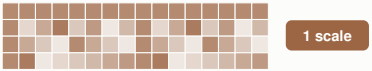
The accounting

Muon now takes 2.1% of an iteration, 165 ms, which puts the iteration at about 7.9 s (derived). The team's communication-bound floor is 55 ms; the gap is stragglers, kernel dispatch, load imbalance and resharding copies. (ZeRO: Rajbhandari et al. 2020.)

Numerics: from tensor-wise FP8 to MXFP8

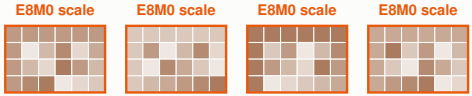
Both models train matrix multiplies in 8-bit floats. Base-2 scales every block of 32 values separately, with no delay.

Base-1: one scale per tensor, from recent history



E4M3 for weights and activations, E5M2 for gradients. The scale lags the data by a step, so a sudden outlier can overflow.

Base-2: one scale per block of 32, computed on the spot



E4M3 everywhere: each block carries an 8-bit power-of-two scale, so range is handled per block and the mantissa bits go to precision.

Background: the formats

FP8 (Micikevicius et al. 2022): E4M3 reaches 448 with 3 mantissa bits, E5M2 57,344 with 2; delayed tensor-wise scaling was standard.

Microscaling (Rouhani et al. 2023; OCP MX): blocks of 32 share an E8M0 scale; native on Blackwell.

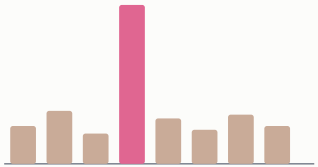
Why it is a stability topic

One tensor-wide scale is set by the largest value, so an outlier squeezes the rest toward zero, and a delayed scale can be wrong for a step. Current, per-block scales remove both failure modes, which is why E4M3 works even for gradients.

Background: global versus activation gradient clipping

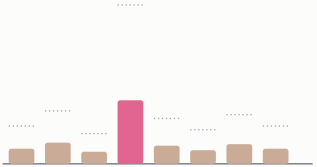
Outlier gradients usually come from a few examples in a batch. Global clipping punishes the whole batch for them.

per-example gradients in one batch



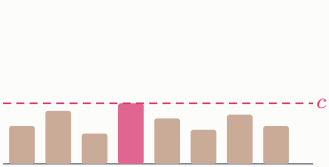
one malformed example dominates the norm

global norm clipping



$g \leftarrow g \cdot \min(1, c/\|g\|)$: every example is scaled down

activation gradient clipping



only the outlier's activation gradients are clipped

Background

Global-norm clipping (Pascanu et al. 2013) is the standard guard against exploding gradients; Base-1 used it alone.

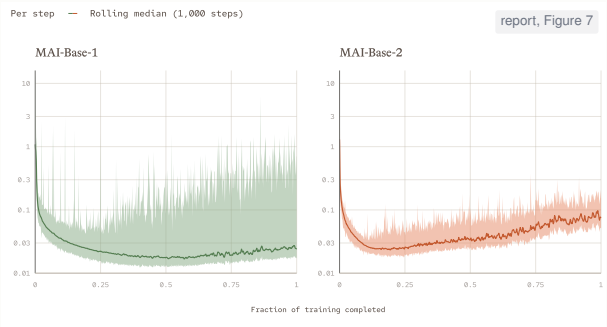
Weight gradients are sums over the batch, so one outlier sets the norm and the rest pay.

What MAI-Base-2 does

Keeps global clipping and adds clipping of activation gradients at selected places. Those are per token, so the clip lands only on the offending positions. With Muon, this tightened the gradient norm; the clip events double as a data-quality detector.

Figure 7: gradient norm, per step and rolling median

Log scale. MAI-Base-1's norm spans two decades with spikes; MAI-Base-2's stays in a narrow band for the whole run.



Base-1. Per-step norms reach 1 to 3 throughout the run, more than a hundred times the median, and the spikes never stop.

Base-2. The band is about a decade wide and the median drifts slowly from 0.02 to under 0.1; the report does not comment on the drift.

Attribution. The report credits activation gradient clipping and Muon jointly; a norm that stays predictable is what lets a 38-day run go without a single rollback (Figure 5).

Why the gradient norm is the quantity to watch

Global clipping fires whenever the norm exceeds its threshold, so a spiky norm means many silently shortened steps; a norm in band means the optimizer sees the step it was tuned for.

Figures 8 and 9: a gain that grows with scale, and a bug it found

Clipping is unusual twice over: its efficiency gain rises with model size, and its clip events point at bad data.

report, Figure 8

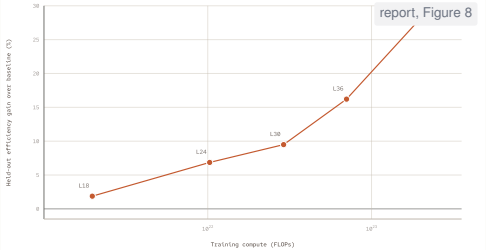


Figure 8. $\mathrm{EG}_{\mathrm{FLOPs}}$ on held-out evaluations from activation gradient clipping, by rung: about $+2\%$ at L18, $+7\%$ at L24, $+10\%$ at L30, $+16\%$ at L36, $+28\%$ at L42 (values read off the plot except $+28\%$). Most recipe changes give a roughly constant gain across scale; this one grows, so a small-rung test understates it.

report, Figure 9

```
/*
 * Copyright 2010 the original author or authors.
 *
 * Licensed under the Apache License, Version 2.0 (the "License");
 * you may not use this file except in compliance with the License.
 * You may obtain a copy of the License at
 *
 *      http://www.apache.org/licenses/LICENSE-2.0
 *
 * Unless required by applicable law or agreed to in writing, software
 * distributed under# PR Title
[pull] master from gradle:master

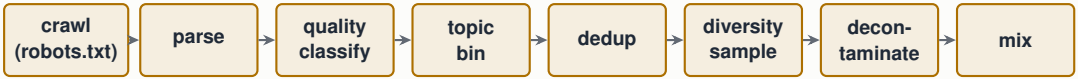
# PR Body
See
```

Figure 9. A pull request from the code data: an Apache license header cut off mid-sentence at “distributed under”, followed by “# PR Title”. The clipped token (boxed #) was the signal. Tracing it back exposed a preprocessing bug, since fixed.

The lesson. A training sequence that forces an extreme gradient is often malformed, so the clipping log is a free data audit.

Scaling data: every token processed in-house

Organic data with provenance, no distillation from other models, and a pipeline that was rebuilt between the two models.



Principles. Organic data, no distillation; every token processed in-house; provenance good enough to license the weights through Foundry.

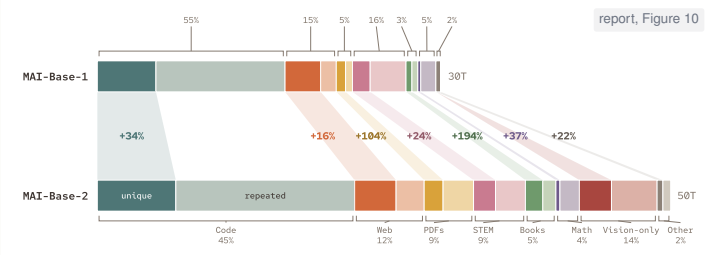
New for Base-2. Much more web content, a March 2026 cutoff, and embedding-based diversity sampling (a substantial gain, per the report). Excluded: sources violating Microsoft's Responsible AI policies or on the USTR Notorious Markets list.

Background: diversity by embedding

SemDeDup (Abbas et al. 2023) embeds documents and removes near-duplicates that exact matching misses; **D4** (Tirumala et al. 2023) goes on to pick a diverse subset and finds it beats random selection at equal tokens. Both reduce the repetition that source scaling (slide 12) has to account for.

Figure 10: the mixture, 30T tokens to 50T

Solid segments are unique tokens, light segments are repeats. Vision data is new; books and PDFs grew most.



Unique tokens grew 36%, the report's own measure of corpus size; the rest of the 30T to 50T increase is deliberate repetition. Code fell from 55% to 45% of the mix but still grew.

Vision. The VLM corpus holds 4.5T unique tokens and, with repetition, 18% of the mixture (9T): about two passes (derived). The 14% vision-only slice is part of that.

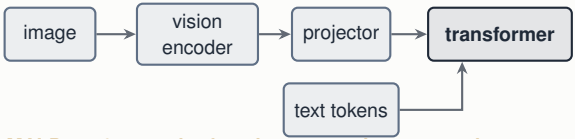
Base-2 share		tokens	vs Base-1
Code	45%	22.5T	+34%
Vision-only	14%	7T	new
Web	12%	6T	+16%
PDFs	9%	4.5T	+104%
STEM	9%	4.5T	+24%
Books	5%	2.5T	+194%
Math	4%	2T	+37%
Other	2%	1T	+22%

shares and growth read off Figure 10; token counts derived from 50T

Native vision without an image encoder

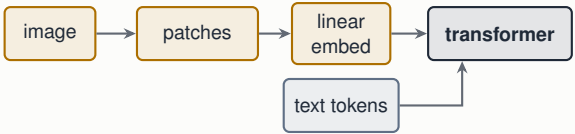
MAI-Base-2 is pre-trained on images from the start, with patches fed into the same transformer as the text.

the common design: a separate vision encoder, bolted on



pre-trained separately (e.g. a CLIP-style ViT), usually added after text pre-training; fixed input resolution

MAI-Base-2: encoder-free, images are just more tokens



one model from the first token; vision and text share every parameter; needs far more vision data, hence 4.5T unique VLM tokens. Precedents: Fuyu-8B (Adept 2023), EVE (Diao et al. 2024)

Corpus (report): image captioning from the web, chart understanding, visual grounding, spatial reasoning, medical understanding, computer use, sheet-music reading.

Part One's components, Part Two's choices

The same block diagram, filled in with what MAI-Base-2 actually does, and the three principles the report closes on.

Part One component	In MAI-Base-2
Tokens, embedding, unembedding	untied input and output tables
Attention	96 query heads; 2 K/V heads in global layers (1 in 6), 16 in local layers
MLP	mixture of 512 experts, 1 shared + 7 routed per token, in every layer
Block, stack	102 blocks, width 8,704; 2.8T parameters total, 80B active
Output, loss	bits per byte on held-out code, QA, STEM and math
Training	50T tokens; Muon + AdamW; MXFP8 E4M3; weight and activation gradient clipping
Hyperparameters	from fitted laws: LR, weight decay, dropout as functions of steps, TPP, size
Scale	16k GB200 GPUs, 38 days, 21% MFU, 89% goodput; $\approx 2.4 \times 10^{25}$ FLOPs

Not stated in the report: positional encoding, normalization placement, vocabulary size, context length.

Learned from first principles
organic data, scaling science, no distillation

Simple methods scale best
one ladder, one recipe, one metric

Rigor over shortcuts
laws fitted, validated, then trusted

Sources

All MAI-Base-2 facts and figures are from the report; background references were checked against their arXiv pages.

The report

Microsoft AI. Frontier Pre-training at Microsoft AI (MAI-Base-2). microsoft.ai, 2026. Figures 1–10 reproduced from it.

Mixture of experts

Shazeer et al. 2017. Outrageously Large Neural Networks (sparsely-gated MoE). arXiv:1701.06538
Lepikhin et al. 2020. GShard. arXiv:2006.16668
Fedus, Zoph, Shazeer 2021. Switch Transformers. arXiv:2101.03961
Gale et al. 2022. MegaBlocks (dropless MoE). arXiv:2211.15841
Dai et al. 2024. DeepSeekMoE. arXiv:2401.06066
DeepSeek-AI 2024. DeepSeek-V3 Technical Report. arXiv:2412.19437

Scaling

Kaplan et al. 2020. Scaling Laws for Neural Language Models. arXiv:2001.08361
Hoffmann et al. 2022. Training Compute-Optimal Large Language Models. arXiv:2203.15556
Muennighoff et al. 2023. Scaling Data-Constrained Language Models. arXiv:2305.16264
Bhagia et al. 2024. Task Scaling Laws via Compute-Efficient Model Ladders. arXiv:2412.04403

Hyperparameters

Loshchilov, Hutter 2019. Decoupled Weight Decay Regularization. arXiv:1711.05101
Wang, Aitchison 2024. How to set AdamW's weight decay as you scale. arXiv:2405.13698
Malladi et al. 2022. SDEs and Scaling Rules for Adaptive Gradient Algorithms. arXiv:2205.10287

Optimizer and systems

Jordan 2024. Muon: an optimizer for hidden layers. kellerjordan.github.io/posts/muon
Bernstein, Newhouse 2024. Old Optimizer, New Norm: An Anthology. arXiv:2409.20325
Liu et al. 2025. Muon is Scalable for LLM Training. arXiv:2502.16982
Kimi Team 2025. Kimi K2: Open Agentic Intelligence. arXiv:2507.20534
Rajbhandari et al. 2020. ZeRO. arXiv:1910.02054
Chowdhery et al. 2022. PaLM (MFU definition). arXiv:2204.02311

Numerics and stability

Micikevicius et al. 2022. FP8 Formats for Deep Learning. arXiv:2209.05433
Rouhani et al. 2023. Microscaling Data Formats for Deep Learning. arXiv:2310.10537
Pascanu, Mikolov, Bengio 2013. On the difficulty of training RNNs (gradient clipping). arXiv:1211.5063
Wortsman et al. 2023. Small-scale proxies for large-scale training instabilities. arXiv:2309.14322

Attention, embeddings, vision, data

Ainslie et al. 2023. GQA. arXiv:2305.13245
Gemma Team 2024. Gemma 2. arXiv:2408.00118
Press, Wolf 2017. Using the Output Embedding to Improve Language Models. arXiv:1608.05859
Bavishi et al. 2023. Fuyu-8B. adept.ai/blog/fuyu-8b
Diao et al. 2024. Unveiling Encoder-Free Vision-Language Models. arXiv:2406.11832
Abbas et al. 2023. SemDeDup. arXiv:2303.09540
Tirumala et al. 2023. D4 (document de-duplication and diversification). arXiv:2308.12284